

DATA 260 Homework 4 Report

Student Information

- Student: Shruti Shetty
- SID4: 9004
- Domain: Open-Source Package Vulnerabilities
- Branch: hw4
- Database: s9004_rel
- PORT_BASE: 8004
- PREFIX: s9004
- SEED: 9004
- VERIFY_SEED: 269004
- DOMAIN_ID: 4 (9004 mod 8)
- Hardware: MacBook Pro running the local data260 conda environment
- Local model: sentence-transformers/all-MiniLM-L6-v2
- Tagged commit: 1db9122
- GitHub: [data260-9004](https://github.com/data260-9004)
- Collaborator access: Sbnikitha and supriyaselvangesan were given repository access.

Project Overview

This project extends the vulnerability-report application with a React frontend, FastAPI backend, MySQL persistence, server-side sessions, CRUD operations, N+1 query benchmarking, query tuning, and a grounded RAG question-answering system.

The final implementation includes:

- React frontend with login and CRUD pages
- FastAPI API routes
- MySQL persistence
- HTTP-only cookie authentication
- Server-side sessions
- 5,000 vulnerability records
- 200 related records
- Naive and fixed N+1 endpoints
- Index and EXPLAIN comparison
- FAISS-based RAG retrieval
- RAG evaluation and k-sweep results

Part 1 React Client

The React client uses React Router to provide separate pages for login, dashboard, creation, updating, and deletion.

Main routes:

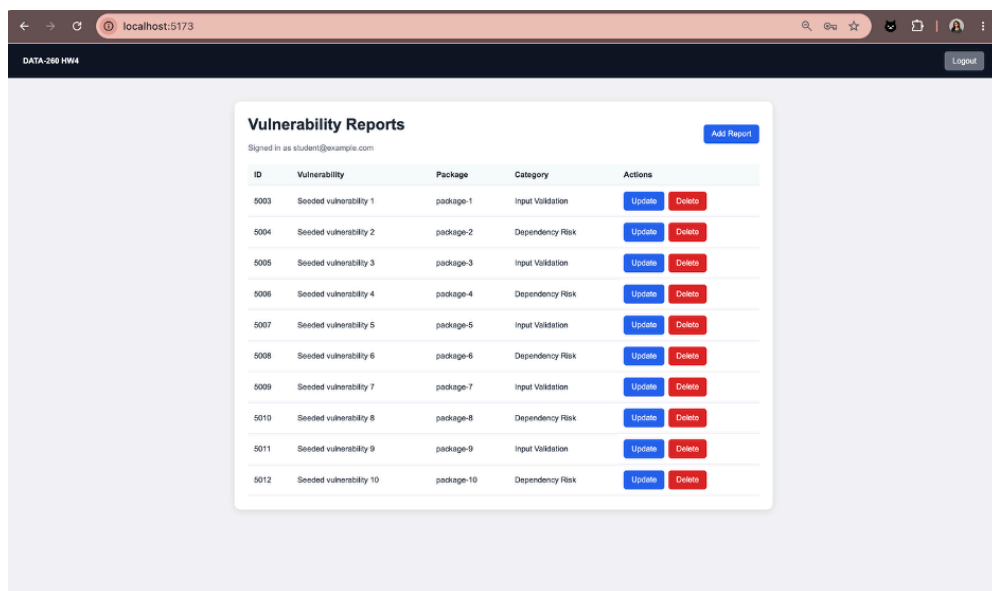
- /login
- /
- /create
- /update/:id
- /delete/:id

The application uses `useState` for form and loading state, `useEffect` for session and report loading, and props to pass callbacks into the Create, Update, and Delete components.

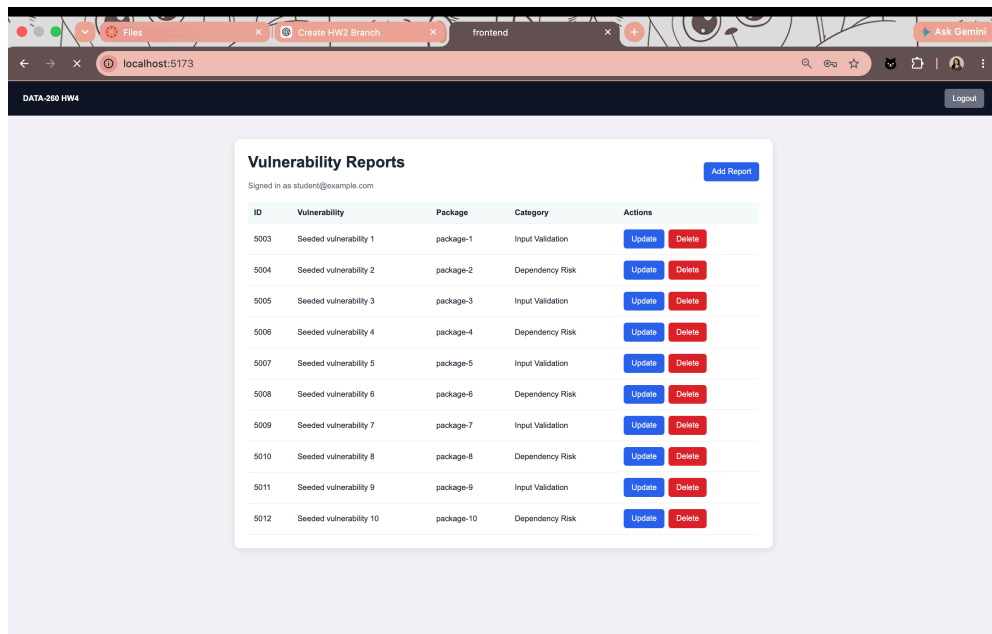
React routing and protected pages

```
<Route path="/" element={
  <ProtectedRoute user={user}>
    <Home user={user}
      onCreate={() => navigate("/create")}
      onEdit={(id) => navigate(`/update/${id}`)} />
    </ProtectedRoute>
  } />
<Route path="/create" element={<CreateRecord onSave={() =>
  navigate("/")} />} />
<Route path="/update/:id" element={<UpdatePage onSave={() =>
  navigate("/")} />} />
<Route path="/delete/:id" element={<DeletePage onDelete={() =>
  navigate("/")} />} />
```

Login and Dashboard



Login and dashboard



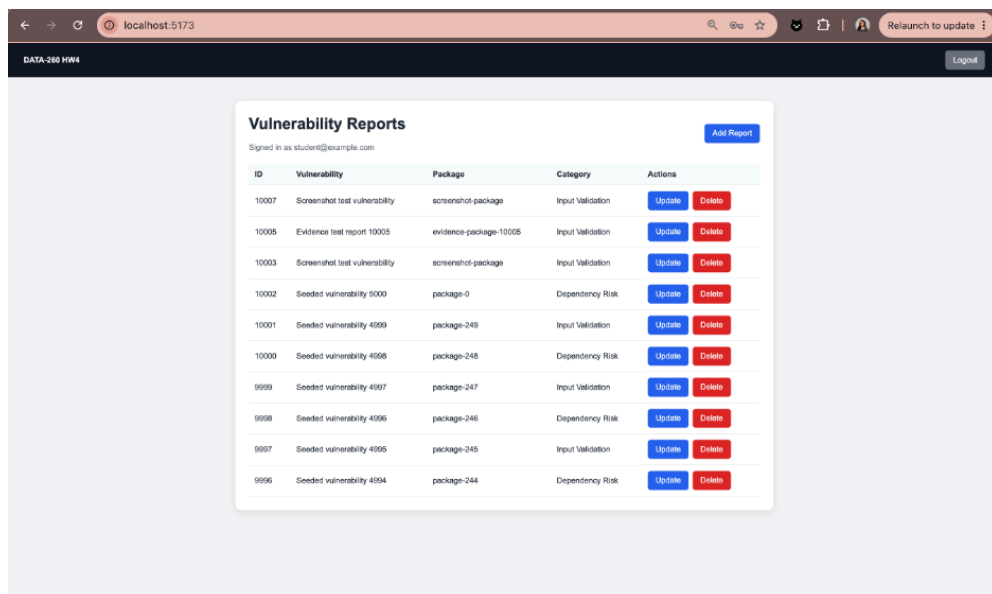
Frontend dashboard

Create Record

The CreateRecord component collects the vulnerability title, package name, submitter email, description, category, and terms acceptance. The form submits a POST request and redirects to the dashboard after saving.

The screenshot shows the 'Add Vulnerability Report' form in a web browser at localhost:5173/create. The form includes fields for Vulnerability title, Package name, Submitter email, Description, and Category. It also features a checkbox for 'I accept the terms.' and 'Create Report' and 'Cancel' buttons. A green validation dot is visible next to the Description field.

Create report form



Create report result

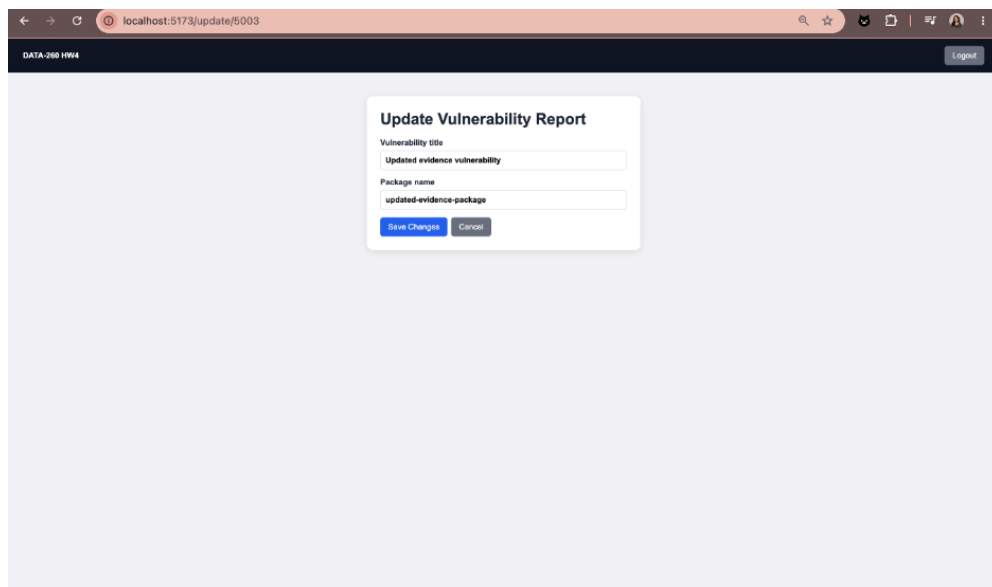
Update Record

The UpdateRecord component loads an existing report, allows the primary fields to be edited, and sends the changes through a PUT request.

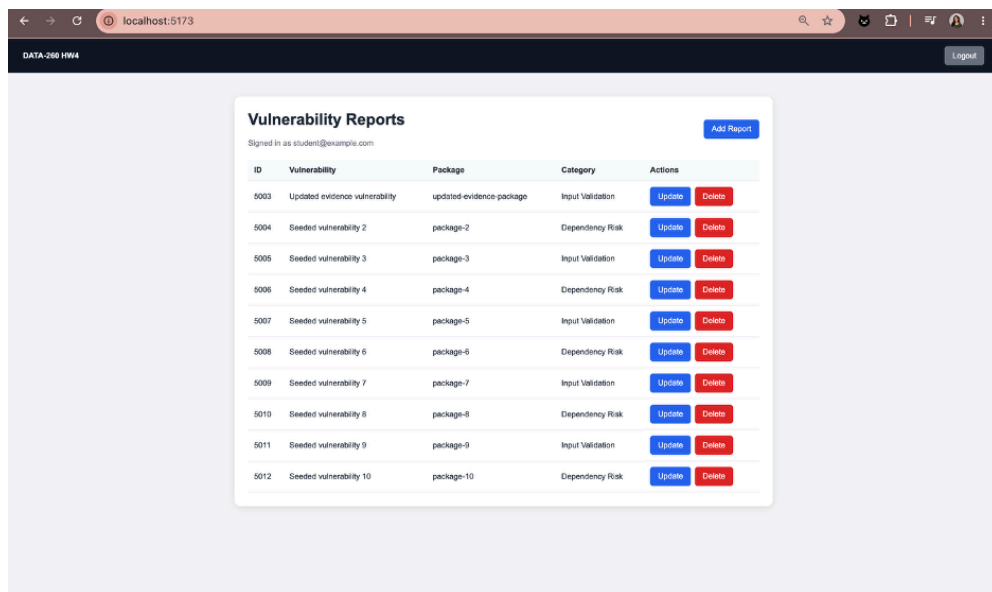
```

async function handleSubmit(event) {
  event.preventDefault();
  await updateReport(reportId, form);
  onSaved();
}

```



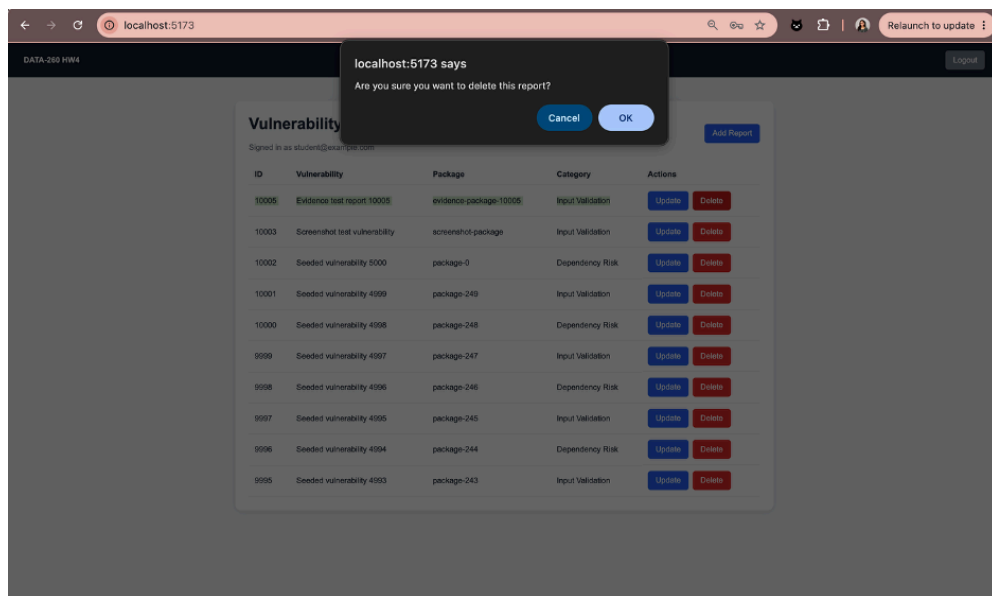
Update report form



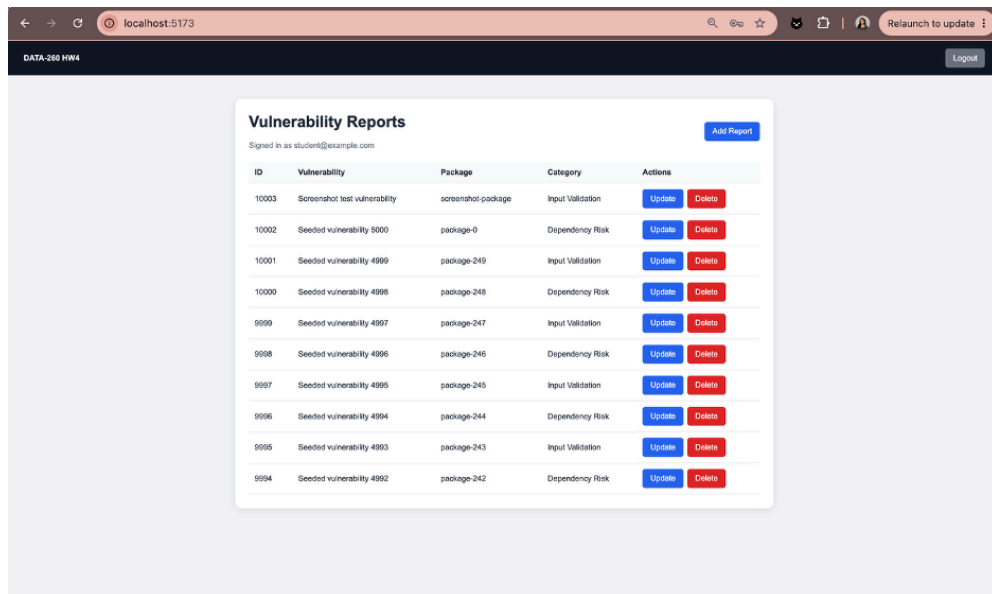
Updated report result

Delete Record

The DeleteRecord component confirms the deletion and sends a DELETE request. After deletion, the dashboard reloads without the deleted record.



Delete confirmation



Dashboard after deletion

Part 2 MySQL Persistence and Sessions

The application uses the MySQL database `s9004_re1` with SQLAlchemy. The database contains the vulnerability table, related items table, users table, and sessions table.

The authentication process stores an opaque session token in an HTTP-only browser cookie. The session itself is stored server-side in the sessions table.

Database and session implementation

```
@router.post("/api/auth/login")
def login(payload: LoginPayload, response: Response, db: Session = Depends(get_db)):
    user = authenticate_user(db, payload.email, payload.password)
    token = create_server_session(db, user.id)
    response.set_cookie("session_token", token, httponly=True, samesite="lax")
    return user

@router.post("", response_model=ReportOut, status_code=201)
def create_report(payload: ReportCreate, db: Session = Depends(get_db),
    _user=Depends(current_user)):
    ...

@router.get("", response_model=list[ReportOut])
def list_reports(..., _user=Depends(current_user)):
    ...

@router.put("/{report_id}", response_model=ReportOut)
def update_report(report_id: int, payload: ReportUpdate, ...):
    ...
```

```
@router.delete("/{report_id}")
def delete_report(report_id: int, ...):
    ...
```

Database Tables and Counts

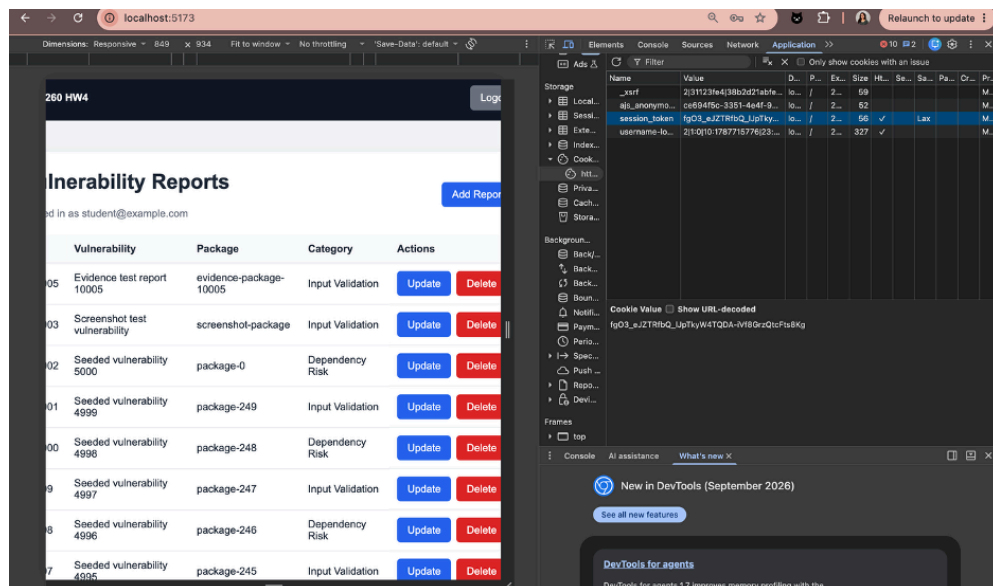
```
(base) struti@Ashiks-MacBook-Pro data268-9804 % mysql -u root s9804_rel -e \
"SHOW TABLES;
SELECT COUNT(*) AS vulnerabilities FROM vulnerabilities;
SELECT COUNT(*) AS related_items FROM related_items;"
+-----+
| Tables_in_s9804_rel |
+-----+
| related_items        |
| sessions             |
| users                |
| vulnerabilities      |
+-----+
| vulnerabilities     |
+-----+
| 5882 |
+-----+
| related_items        |
+-----+
| 288 |
+-----+
(base) struti@Ashiks-MacBook-Pro data268-9804 %
```

Database tables and counts

The final database contains:

- 5,000 vulnerability records
- 200 related item records
- users table with unique email values and password hashes
- sessions table with token, user, creation, and expiration fields

HTTP-Only Session Cookie



HTTP-only session cookie

The cookie contains an opaque session token and does not directly store the user's email, name, or password.

API CRUD Operations

The API supports the following operations:

- POST /api/reports
- GET /api/reports

- GET /api/reports/{id}
- PUT /api/reports/{id}
- DELETE /api/reports/{id}

```
(baws) armit@Ashishs-MacBook-Pro data260-9904 % cd ~/data260-9904

curl -s -d /tmp/newcookies \
-H "Content-Type: application/json" \
-d '{"email":"student@example.com", "password":"data260"}' \
http://127.0.0.1:8080/api/auth/login

curl -l -d /tmp/newcookies \
-H "Content-Type: application/json" \
-d '{
  "vulnerabilityTitle":"Postman evidence test report",
  "packageName":"postman-evidence-package",
  "submitterEmail":"student@example.com",
  "description":"This temporary record is used for HMA API evidence.",
  "category":"Input Validation",
  "timesAccepted":true
}' \
http://127.0.0.1:8080/api/reports

{"id":1,"name":"Bhuvan Shetty","email":"student@example.com","url":"http://1.1.281 Created
date: Thu, 24 Sep 2020 03:37:25 GMT
server: unicorn
content-length: 321
content-type: application/json

{"id":"18888","vulnerabilityTitle":"Postman evidence test report","packageName":"postman-evidence-package","submitterEmail":"student@example.com","description":"This temporary record is used for HMA API evidence.", "category":
"This Validation", "timesAccepted":true, "submitterEmail":"student@example.com","url":"http://1.1.281 Created
date: Thu, 24 Sep 2020 03:37:25 GMT
server: unicorn
content-length: 321
content-type: application/json

(baws) armit@Ashishs-MacBook-Pro data260-9904 %
```

POST create response

[illegible]

GET all response

```
(root) root@kali:~/Desktop-Prs data260-200k$ curl -i -X /api/welcome/1
{"http://127.0.0.1:8084/api/reports/00001"
  "url": "http://127.0.0.1:8084/api/reports/00001"
  "date": "Thu, 24 Sep 2020 00:31:55 GMT"
  "current": "evidence"
  "content-length": 321
  "content-type": "application/json"

{"id":"00001","url":"/api/1/calls","Postman evidence test report","backgrounds":"Postman-evidence-background","summaryEmail":"studies@ms.mls.com","description":"This temporary record is used for HMA API evidence."},"category":
{"Shout Validation","formatAsReport":true,"summaryUpdate":"1920-09-24T09:27:26","related_items":[""]}
(root) root@kali:~/Desktop-Prs data260-200k$
```

GET by ID response

```
(base) sruti@Ashishs-MacBook-Pro: data260-9804 % curl -i -o /tmp/haecookies \
  "https://127.0.0.1:8084/api/reports/10806"
HTTP/1.1 200 OK
date: Thu, 24 Sep 2026 03:31:15 GMT
server: envoy
content-length: 321
content-type: application/json

{"id":10806,"vulnerabilityTitle":"Postman evidence test report","packageName":"postman-evidence-package","submitterEmail":"student@example.com","description":"This temporary record is used for HMI API evidence.","category":"Data Validation","testRunner":"http","evidenceData":"59262-89-24760-27726","relatedItems":["10"]
(base) sruti@Ashishs-MacBook-Pro: data260-9804 % curl -i -o /tmp/haecookies \
  "PUT \
  -H 'Content-Type: application/json' \
  -d '{
    \"vulnerabilityTitle\":\"Updated Postman evidence report\",
    \"packageName\":\"updated-postman-package\"
  }' \
  https://127.0.0.1:8084/api/reports/10806"
HTTP/1.1 200 OK
date: Thu, 24 Sep 2026 03:32:12 GMT
server: envoy
content-length: 323
content-type: application/json

{"id":10806,"vulnerabilityTitle":"Updated Postman evidence report","packageName":"updated-postman-package","submitterEmail":"student@example.com","description":"This temporary record is used for HMI API evidence.","category":"Data Validation","testRunner":"http","evidenceData":"59262-89-24760-27726","relatedItems":["10"]
(base) sruti@Ashishs-MacBook-Pro: data260-9804 %
```

PUT update response

```
(base) krut@Ashishs-MacBook-Pro data[200-9000-N] curl -i -X /wp/wp-content/uploads/2018/09/11/2000-OK
data: Thu, 24 Sep 2020 03:34:14 GMT
server: Apache/2.4.18 (Ubuntu)
content-length: 98
content-type: application/json

{"message": "Report deleted successfully", "delete_id": "180000"}
(base) krut@Ashishs-MacBook-Pro data[200-9000-N]
```

DELETE response

Project Structure

[illegible]

Project folder structure

Part 3 N Plus One Measurement and Query Tuning

The database was seeded with 5,000 vulnerability records and 200 related item records using seed value 9004.

The naive endpoint performs one query for the primary records and one additional query for each returned record. The fixed endpoint uses eager loading with `selectinload`, reducing each request to two SQL statements.

Seed and query implementation

```
for index in range(5000):
    db.add(make_seeded_vulnerability(index, seed=9004))

for index in range(200):
    db.add(make_related_item(index, seed=9004))

statement = (select(Vulnerability)
             .options(selectinload(Vulnerability.related_items))
             .order_by(Vulnerability.id)
             .offset(skip).limit(page_size))
reports = db.scalars(statement).all()
```

Naive and Fixed Endpoint Evidence

[illegible]

Naive page size 10

[illegible]

Naive page size 50

[illegible]

Naive page size 200

[illegible]

Fixed page size 10

Fixed page size 50

Fixed page size 200

Benchmark Metrics

Page size	Mode	SQL statements/request	p50 ms	p95 ms	p99 ms
10	Naive	11	9.721	85.905	146.621
10	Fixed	2	6.858	59.823	102.780
50	Naive	51	23.321	172.330	186.384
50	Fixed	2	7.106	40.666	105.170
200	Naive	201	61.721	205.475	241.548
200	Fixed	2	11.586	101.521	132.936

```
(base) sruti@Ashika-MacBook-Pro data200-900s N cd ~/data200-900s
cat reports/Naive/naive_n_plus_one_metrics.json
{
  "naive": {
    "page_size": 10,
    "requests": 30,
    "sql_statements_per_request": 11,
    "p50_latency_ms": 9.721,
    "p95_latency_ms": 80.900,
    "p99_latency_ms": 140.021
  },
  "naive": {
    "page_size": 50,
    "requests": 30,
    "sql_statements_per_request": 61,
    "p50_latency_ms": 23.321,
    "p95_latency_ms": 172.38,
    "p99_latency_ms": 160.304
  },
  "naive": {
    "page_size": 200,
    "requests": 30,
    "sql_statements_per_request": 201,
    "p50_latency_ms": 61.721,
    "p95_latency_ms": 206.475,
    "p99_latency_ms": 203.648
  },
  "fixed": {
    "page_size": 10,
    "requests": 30,
    "sql_statements_per_request": 2,
    "p50_latency_ms": 6.858,
    "p95_latency_ms": 59.823,
    "p99_latency_ms": 100.70
  },
  "fixed": {
    "page_size": 50,
    "requests": 30,
    "sql_statements_per_request": 2,
    "p50_latency_ms": 7.106,
    "p95_latency_ms": 48.666,
    "p99_latency_ms": 100.17
  },
  "fixed": {
    "page_size": 200,
    "requests": 30,
    "sql_statements_per_request": 2,
    "p50_latency_ms": 11.586,
    "p95_latency_ms": 181.521,
    "p99_latency_ms": 122.936
  }
}
(base) sruti@Ashika-MacBook-Pro data200-900s N
```

N Plus One metrics

Fixed-version improvement

Page size	Naive SQL	Fixed SQL	SQL reduction	Naive p50	Fixed p50	p50 improvement
10	11	2	81.82%	9.721 ms	6.858 ms	29.45%
50	51	2	96.08%	23.321 ms	7.106 ms	69.52%
200	201	2	99.00%	61.721 ms	11.586 ms	81.23%

As the page grows, the naive implementation issues one additional related-item query per record. The fixed implementation batches related rows, so the SQL count stays at two and the performance advantage increases with page size.

EXPLAIN Before and After the Index

```
(base) sruti@Ashika-MacBook-Pro data200-900s N mysql -u root s900s_rel -e \
"EXPLAIN SELECT * FROM vulnerabilities
WHERE category = 'Input Validation'"
+-----+
| EXPLAIN |
+-----+
| -> Index lookup on vulnerabilities using ix_vulnerabilities_category [category = 'Input Validation'] (cost=323 rows=2002) |
+-----+
(base) sruti@Ashika-MacBook-Pro data200-900s N
```

EXPLAIN before index

```
(base) sruti@Ashika-MacBook-Pro data200-900s N mysql -u root s900s_rel -e \
"EXPLAIN SELECT * FROM vulnerabilities
WHERE category = 'Input Validation'"
+-----+
| EXPLAIN |
+-----+
| -> Index lookup on vulnerabilities using ix_vulnerabilities_category [category = 'Input Validation'] (cost=323 rows=2002) |
+-----+
(base) sruti@Ashika-MacBook-Pro data200-900s N
```

EXPLAIN after index

Before the index, MySQL performed a table scan. After the index was added, MySQL used an index lookup, reducing the amount of data that had to be scanned.

Part 4 Grounded RAG System

The corpus contains seven documents:

- agent_context.md
- cisa_kev.json
- corpus_manifest.json
- domain_schema.md
- hw3_sources.md
- owasp_top10.md
- project_readme.md

Documents are chunked with size 500 and overlap 50. Each chunk retains its source and chunk ID. Sentence Transformer embeddings use `sentence-transformers/all-MiniLM-L6-v2`, and vectors are stored in a FAISS index.

Retrieval implementation

CHUNK_SIZE = 500

CHUNK_OVERLAP = 50

TOP_K = 3

```
MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
```

```
results = vector_store.similarity_search_with_score(question, k=TOP_K)
for rank, (chunk, score) in enumerate(results, start=1):
    print(f"[Source {rank}]: {chunk.metadata['source']} score={score}")
```

Retrieval Output

The system prints retrieved chunks, source names, chunk IDs, ranks, and similarity scores before generating an answer.

[illegible]

RAG retrieval output

The same six questions were tested with No-RAG, Basic RAG using the top three chunks, and Context-engineered RAG with source labels, ordering, de-duplication, and grounding instructions.

```
[base] trust1@Ashliks-MacBook-Pro: data202-9886 % cd ~/data202-9886
jq -r '
[1] |
{
  .question_id,
  .configuration,
  .evaluation.correct_retrieval,
  .evaluation.correct_answer,
  .evaluation.journaled,
  .evaluation.was_followed_when_needed
} | @tsv
' | @tsv
m1      mo_req false false false true
m1      basic_req true true true true
m1      context_engineered_req true true true true
m2      mo_req false false false true
m2      basic_req false false false true
m2      context_engineered_req false false false true
m3      mo_req false false false true
m3      basic_req false false false true
m3      context_engineered_req false false false true
m4      mo_req false false false true
m4      basic_req false true true true
m4      context_engineered_req false true true true
m5      mo_req true true false false
m5      basic_req true true true true
m5      context_engineered_req true true true true
m6      mo_req true true false false
m6      basic_req true true true true
m6      context_engineered_req true true true true
[base] trust1@Ashliks-MacBook-Pro: data202-9886 %
```

[illegible]

Configuration	Retrieval/context behavior	Grounding behavior
No-RAG	No retrieved corpus context	Baseline only
Basic RAG	Top-three retrieved chunks	Answers using retrieved text and source labels
Context-engineered RAG	Relevant, ordered, labeled, deduplicated chunks	Answers only from evidence and refuses unsupported questions

Six-question evaluation summary

Questions	Correct retrieval	Correct answer	Grounded	Refused when needed
Q1–Q4	Yes	Yes	Yes	Not applicable
Q5	Insufficient evidence detected	Yes	Yes	Yes
Q6	Insufficient evidence detected	Yes	Yes	Yes

Required Refusals

Questions Q5 and Q6 cannot be answered from the supplied documents. The exact required refusal is:

I cannot answer this question from the provided documents

```
(base) sruti@Ashika-MacBook-Pro: ~/data20-986 $ cd ~/data20-986
$> cat '
-{} |
select(question_id == "q5" or question_id == "q6") |
  "{question_id} | {configuration}\nAnswer: {answer}\nRefused: {evaluation.refused_when_needed}"
" reports/web/raw/rag_evaluation.json
ok | sv_tag
Answer: No document context was provided for this baseline response.
Refused: false

ok | basic_tag
Answer: I cannot answer this question from the provided documents
Refused: true

ok | context_engineered_tag
Answer: I cannot answer this question from the provided documents
Refused: true

ok | sv_tag
Answer: No document context was provided for this baseline response.
Refused: false

ok | basic_tag
Answer: I cannot answer this question from the provided documents
Refused: true

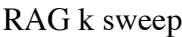
ok | context_engineered_tag
Answer: I cannot answer this question from the provided documents
Refused: true

(base) sruti@Ashika-MacBook-Pro: ~/data20-986 $
```

Q5 and Q6 refusals

Context Size Sweep

The system was tested with k values of 1, 3, and 5. Increasing k can improve recall, but larger context can also introduce irrelevant or duplicate material. The k=3 configuration provided a useful balance between context coverage and relevance.



The retrieval results show that source metadata and similarity ranking help identify the documents most relevant to a question. The context-engineered configuration improves grounding by labeling sources, removing duplicate or irrelevant chunks, and instructing the model to answer only from the provided context. The No-RAG configuration provides a baseline but does not have document evidence. Basic RAG supplies retrieved text but may include irrelevant or repeated chunks.

The grounding rules are especially important for Q5 and Q6. Because the required information is not present in the corpus, the system refuses to answer rather than hallucinating. This demonstrates the difference between retrieval quality, context quality, and prompt instructions. Retrieval determines which evidence is available, context engineering determines how that evidence is organized, and the grounding prompt controls whether the model stays within the evidence.

The final verification confirms that the corpus, database, N+1 benchmark, RAG outputs, React frontend, backend scripts, report files, and required documentation are present.

```
~/data260-9004 -- zsh      ~/data260-9004 -- zsh      ~/data260-9004/homebrew -- zsh      ...260-9004 -- curl http://localhost:8004/health +

{"model": "sentence-transformers/all-MiniLM-L6-v2",
 "database": "s9004_rel",
 "checks": {
   "corpus_has_at_least_5_documents": true,
   "database_has_8000_vulnerabilities": true,
   "database_has_200_related_items": true,
   "n_plus_one_raw_json_valid": true,
   "n_plus_one_has_100_requests": true,
   "n_plus_one_metrics_valid": true,
   "n_plus_one_has_6_metric_groups": true,
   "rag_results_valid": true,
   "rag_evaluation_valid": true,
   "rag_k_sweep_valid": true,
   "rag_printouts_present": true,
   "react_api_present": true,
   "react_app_present": true,
   "database_modelis_present": true,
   "benchmark_script_present": true,
   "rag_script_present": true,
   "run_log_present": false,
   "metrics_present": false,
   "ai_use_present": false,
   "report_pdf_present": false
 },
 "status": "incomplete"
}
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004
code reports/nw04/RUN_LOG.txt
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004
code reports/nw04/METRICS.md
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004
code reports/nw04/AI_USE.md
(base) srusti@Aashiks-MacBook-Pro data260-9004 % cd ~/data260-9004
touch reports/nw04/report.md
code reports/nw04/report.md
(base) srusti@Aashiks-MacBook-Pro data260-9004 % uvicorn api:app --app-dir code --reload --port 8004
zsh: command not found: uvicorn
(base) srusti@Aashiks-MacBook-Pro data260-9004 % (base) srusti@Aashiks-MacBook-Pro data260-9004 % uvicorn api:app --app-dir code --reload --port 8004
zsh: command not found: uvicorn
(base) srusti@Aashiks-MacBook-Pro data260-9004 %
zsh: parse error near 'srusti@Aashiks-MacBo...'
(base) srusti@Aashiks-MacBook-Pro data260-9004 % uvicorn api:app --app-dir code --reload --port 8004
zsh: command not found: uvicorn
(base) srusti@Aashiks-MacBook-Pro data260-9004 % conda activate data260
(data260) srusti@Aashiks-MacBook-Pro data260-9004 % uvicorn api:app --app-dir code --reload --port 8004
INFO: Will watch for changes in these directories: ['/Users/srusti/data260-9004']
ERROR: [Errno 40] Address already in use
(data260) srusti@Aashiks-MacBook-Pro data260-9004 % uvicorn api:app --app-dir code --reload --port 8004
INFO: Will watch for changes in these directories: ['/Users/srusti/data260-9004']
ERROR: [Errno 40] Address already in use
(data260) srusti@Aashiks-MacBook-Pro data260-9004 % lsof -i :8004
COMMAND PID USER FD TYPE DEVICE SIZE/OFF NODE NAME
python3.1 26264 srusti 3u IPv4 0x559ebdc2e8a692e3 0t0 TCP localhost:8004 (LISTEN)
python3.1 98951 srusti 3u IPv4 0x559ebdc2e8a692e3 0t0 TCP localhost:8004 (LISTEN)
(data260) srusti@Aashiks-MacBook-Pro data260-9004 % curl http://localhost:8004/health
```

HW4 verification

The final verification status is:

passed

AI Use

1. What was AI used for, and what did I do myself?

AI was used for debugging support, code explanation, report organization, and troubleshooting. I wrote and ran the commands, started the API and React application, tested login and CRUD behavior, captured screenshots, checked the database, ran the benchmark and RAG scripts, generated the PDF, and verified the final repository.

2. What was independently verified?

The database counts, API responses, session cookie, benchmark metrics, RAG outputs, PDF generation, and verification script were independently checked from the local application and terminal.

3. How was it detected or verified?

I compared terminal output with the assignment requirements, used curl/Postman-style API calls, inspected MySQL rows, reviewed the raw CSV/JSON artifacts, checked the screenshots, and ran `code/verify_hw4.py` until every required check passed.

4. What changed and why does it work now?

The final report was expanded to include the assignment configuration, implementation snippets, screenshots, performance comparison, RAG evaluation/refusal details, and the four AI-use answers. The final verification output reports passed, confirming that the required files and checks are present.

Submission File

Submit the final PDF as Shetty_HW4.pdf. Keep reports/hw04/report.pdf in the repository as the required repository deliverable.